JAVA GRAPH ALGORITHMS

Global Tech Interview Preparation • IntelliJ IDEA format • Rules • Formulas • Complete Java implementations • Dry runs

Coverage map

• Graph fundamentals, representations, input templates, constraints and pattern recognition

• BFS, DFS, connected components, grid traversal, cycle detection, bipartite checking

• Topological sorting, DAG shortest paths, course scheduling, DSU and connectivity

• Kruskal, Prim, Dijkstra, Bellman–Ford, Floyd–Warshall, 0–1 BFS and shortest-path variants

• Bridges, articulation points, strongly connected components, Eulerian paths/circuits

• High-frequency implementation problems: islands, rotting oranges, word ladder, clone graph, network delay, cheapest flights, redundant connection, accounts merge and more

Scope honesty: Graph theory is enormous; no PDF can contain every graph algorithm ever published or every interview question. This guide covers the major globally used interview patterns and provides reusable implementations. Advanced topics such as max-flow, matching and computational geometry are listed as further study where appropriate.

IntelliJ format: Each code block is a complete Java class or a clearly identified method/template. Create a Java project, place each public class in its matching .java file, or paste a single class into Main.java after renaming the class. Java version: 17+ recommended.

1. Graph fundamentals: definitions and notation

Graph: A graph is a structure G=(V,E), where V is the set of vertices (nodes) and E is the set of edges connecting pairs of vertices.

Directed graph: u→v is not automatically v→u. **Undirected graph:** edge {u,v} works both ways. **Weighted graph:** every edge has a weight/cost.

Degree: undirected degree = number of incident edges. Directed graphs have indegree (incoming) and outdegree (outgoing).

Path: sequence of connected vertices. **Cycle:** path that returns to its start. **Connected component:** maximal connected group in an undirected graph.

Tree: connected, undirected, acyclic graph. For V vertices, a tree has V−1 edges. A forest is a disjoint union of trees.

Complexity symbols

V = number of vertices, E = number of edges, R×C = grid dimensions. With adjacency lists, BFS/DFS are O(V+E); with an adjacency matrix, scanning neighbors costs $O(V^2)$.

Choose representation

Representation	Space	Best use
Adjacency list	O(V+E)	Sparse graphs; most interviews
Adjacency matrix	$O(V^2)$	Dense graphs; O(1) edge lookup
Edge list	O(E)	Kruskal, Bellman–Ford
Grid as implicit graph	O(RC)	Islands, shortest path in matrix

Input format rules

Most coding platforms provide n,m followed by m edges. Confirm whether vertices are 0-indexed (0..n−1) or 1-indexed (1..n). Convert 1-indexed input with u-- and v--. For an undirected graph, add both directions. For a directed graph, add only u→v.

Java adjacency-list template

```
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());

// Undirected edge u-v:
graph.get(u).add(v);
graph.get(v).add(u);

// Directed edge u -> v:
graph.get(u).add(v);
```

How to identify the graph pattern

• Minimum number of edges/hops in an unweighted graph → BFS.

• Explore all reachable nodes / count components → DFS or BFS.

• Dependency order / prerequisites → Topological sort (DAG only).

• Cycle in undirected graph → DFS parent check or DSU.

• Cycle in directed graph → DFS recursion-state or topological sort.

• Shortest path with non-negative weights → Dijkstra.

• Negative edge weights → Bellman–Ford; all-pairs small/dense graph → Floyd–Warshall.

• Connect all vertices at minimum total edge cost → MST (Kruskal/Prim).

• Connectivity after repeated unions → DSU.

• Critical edge / vertex whose removal disconnects graph → Bridge / articulation point.

• Directed mutual reachability groups → SCC (Kosaraju/Tarjan).

• Grid movement → model each cell as a vertex and legal moves as edges.

2. IntelliJ reusable templates

2.1 Complete BFS traversal

What it solves: Visit vertices in increasing number of edges from a source.

Recognition rule / invariant: Mark visited when enqueuing, not when dequeuing; this prevents duplicate queue entries.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class BFS {
    public static List<Integer> bfs(List<List<Integer>> g, int source) {
        int n = g.size();
        boolean[] visited = new boolean[n];
        Queue<Integer> q = new ArrayDeque<>();
        List<Integer> order = new ArrayList<>();

        visited[source] = true;
        q.offer(source);

        while (!q.isEmpty()) {
            int u = q.poll();
            order.add(u);
            for (int v : g.get(u)) {
                if (!visited[v]) {
                    visited[v] = true;
                    q.offer(v);
                }
            }
        }
        return order;
    }
}
```

Example / dry run: 0—1—2 and 0—3: BFS from 0 visits 0, then 1 and 3, then 2 (exact order between 1 and 3 depends on adjacency order).

Important: One BFS from a source does not visit disconnected components. Run BFS from every unvisited vertex to cover the whole graph.

2.2 Complete DFS traversal (recursive)

What it solves: Explore as deeply as possible before backtracking.

Recognition rule / invariant: Mark a node visited before exploring neighbors. The visited array prevents infinite recursion on cycles.

Complexity: Time O(V+E), space O(V) including recursion stack.

IntelliJ-ready Java implementation

```
import java.util.*;

public class DFSRecursive {
    static void dfs(int u, List<List<Integer>> g, boolean[] seen,
                    List<Integer> order) {
        seen[u] = true;
        order.add(u);
        for (int v : g.get(u)) {
            if (!seen[v]) dfs(v, g, seen, order);
        }
    }

    public static List<Integer> traverse(List<List<Integer>> g, int source) {
        boolean[] seen = new boolean[g.size()];
        List<Integer> order = new ArrayList<>();
        dfs(source, g, seen, order);
        return order;
    }
}
```

Example / dry run: DFS order depends on neighbor insertion order. For very deep graphs, recursive DFS may cause StackOverflowError; use iterative DFS when needed.

Important: Recursion depth can be O(V). For large constraints, prefer iterative DFS.

2.3 Iterative DFS

What it solves: DFS without recursion, avoiding Java call-stack overflow.

Recognition rule / invariant: Push source; pop a vertex; if unseen, mark and push its unvisited neighbors.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class DFSIterative {
    static List<Integer> dfs(List<List<Integer>> g, int source) {
        boolean[] seen = new boolean[g.size()];
        Deque<Integer> st = new ArrayDeque<>();
        List<Integer> order = new ArrayList<>();
        st.push(source);

        while (!st.isEmpty()) {
            int u = st.pop();
            if (seen[u]) continue;
            seen[u] = true;
            order.add(u);
            for (int i = g.get(u).size() - 1; i >= 0; i--) {
                int v = g.get(u).get(i);
                if (!seen[v]) st.push(v);
            }
        }
        return order;
    }
}
```

Example / dry run: Stack-based traversal order may differ from recursive DFS. Reverse neighbor iteration if matching recursive order is important.

Important: Mark-on-pop may push duplicates; mark-on-push is also valid if you want to avoid duplicate stack entries.

2.4 Count connected components

What it solves: Count disconnected groups in an undirected graph.

Recognition rule / invariant: Start a traversal from each unvisited vertex; each new traversal identifies one component.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class ConnectedComponents {
    static int count(List<List<Integer>> g) {
        int n = g.size(), components = 0;
        boolean[] seen = new boolean[n];
        for (int i = 0; i < n; i++) {
            if (!seen[i]) {
                components++;
                Queue<Integer> q = new ArrayDeque<>();
                q.offer(i);
                seen[i] = true;
                while (!q.isEmpty()) {
                    int u = q.poll();
                    for (int v : g.get(u)) {
                        if (!seen[v]) {
                            seen[v] = true;
                            q.offer(v);
                        }
                    }
                }
            }
        }
        return components;
    }
}
```

Example / dry run: Vertices {0,1,2} connected and {3,4} connected form 2 components.

Important: Component counting normally refers to undirected graphs. Directed graphs require a definition such as weakly connected or strongly connected.

2.5 Shortest path in an unweighted graph

What it solves: Return minimum edge count from source to every vertex.

Recognition rule / invariant: BFS distance: dist[v] = dist[u] + 1 on first discovery.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class UnweightedShortestPath {
    static int[] distances(List<List<Integer>> g, int src) {
        int[] dist = new int[g.size()];
        Arrays.fill(dist, -1);
```

```
        Queue<Integer> q = new ArrayDeque<>();
        dist[src] = 0;
        q.offer(src);

        while (!q.isEmpty()) {
            int u = q.poll();
            for (int v : g.get(u)) {
                if (dist[v] == -1) {
                    dist[v] = dist[u] + 1;
                    q.offer(v);
                }
            }
        }
        return dist;
    }
}
```

Example / dry run: If path 0→2→4 uses 2 edges, dist[4]=2. Unreachable vertices remain -1.

Important: Use BFS only when every edge has equal cost (or the graph is unweighted).

3. Grid graph problems

3.1 Number of Islands

What it solves: Count connected groups of land cells ('1') in a 2D grid using 4-direction movement.

Recognition rule / invariant: Each unvisited land cell starts one island; flood-fill and mark the entire island.

Complexity: Time O(RC), space O(RC) worst case for queue.

IntelliJ-ready Java implementation

```
import java.util.*;

public class NumberOfIslands {
    static int numIslands(char[][] grid) {
        if (grid == null || grid.length == 0) return 0;
        int r = grid.length, c = grid[0].length, count = 0;
        int[][] d = {{1,0},{-1,0},{0,1},{0,-1}};
        for (int i=0;i<r;i++) for (int j=0;j<c;j++) {
            if (grid[i][j] == '1') {
                count++;
                Queue<int[]> q = new ArrayDeque<>();
                q.offer(new int[]{i,j});
                grid[i][j] = '0';
                while (!q.isEmpty()) {
                    int[] x=q.poll();
                    for (int[] z:d) {
                        int ni=x[0]+z[0], nj=x[1]+z[1];
                        if (ni>=0 && ni<r && nj>=0 && nj<c &&
                            grid[ni][nj]=='1') {
                            grid[ni][nj]='0';
                            q.offer(new int[]{ni,nj});
                        }
                    }
                }
            }
        }
        return count;
    }
}
```

Example / dry run: Grid with one island shaped as a block and another separate land cell returns 2.

Important: This implementation modifies the grid. Clone it first if the original must be preserved. Use 8 directions only if the statement says diagonal adjacency counts.

3.2 Flood Fill

What it solves: Recolor the entire 4-direction connected region matching the starting cell's original color.

Recognition rule / invariant: Remember originalColor; visit only cells equal to it. If originalColor equals newColor, return immediately.

Complexity: Time O(RC), space O(RC).

IntelliJ-ready Java implementation

```
import java.util.*;

public class FloodFill {
    static int[][] floodFill(int[][] image, int sr, int sc, int color) {
        int old = image[sr][sc];
        if (old == color) return image;
        int r=image.length, c=image[0].length;
```

```
        int[][] d={{1,0},{-1,0},{0,1},{0,-1}};
        Queue<int[]> q=new ArrayDeque<>();
        q.offer(new int[]{sr,sc});
        image[sr][sc]=color;
        while(!q.isEmpty()){
            int[] x=q.poll();
            for(int[] z:d){
                int nr=x[0]+z[0], nc=x[1]+z[1];
                if(nr>=0 && nr<r && nc>=0 && nc<c && image[nr][nc]==old){
                    image[nr][nc]=color;
                    q.offer(new int[]{nr,nc});
                }
            }
        }
        return image;
    }
}
```

Example / dry run: Starting at a pixel colored 1, all connected 1s become 2; disconnected 1s stay unchanged.

Important: Mark a cell when enqueuing to avoid adding it repeatedly.

3.3 Rotting Oranges (multi-source BFS)

What it solves: Find minimum minutes until all fresh oranges rot; rotten oranges spread to adjacent fresh oranges each minute.

Recognition rule / invariant: Initialize BFS with every rotten orange at time 0. Process all sources together level by level.

Complexity: Time O(RC), space O(RC).

IntelliJ-ready Java implementation

```
import java.util.*;

public class RottingOranges {
    static int orangesRotting(int[][] a) {
        int r=a.length, c=a[0].length, fresh=0, minutes=0;
        Queue<int[]> q=new ArrayDeque<>();
        int[][] d={{1,0},{-1,0},{0,1},{0,-1}};
        for(int i=0;i<r;i++) for(int j=0;j<c;j++){
            if(a[i][j]==2) q.offer(new int[]{i,j});
            else if(a[i][j]==1) fresh++;
        }
        while(!q.isEmpty() && fresh>0){
            int size=q.size();
            for(int k=0;k<size;k++){
                int[] x=q.poll();
                for(int[] z:d){
                    int nr=x[0]+z[0], nc=x[1]+z[1];
                    if(nr>=0&&nr<r&&nc>=0&&nc<c&&a[nr][nc]==1){
                        a[nr][nc]=2; fresh--;
                        q.offer(new int[]{nr,nc});
                    }
                }
            }
            minutes++;
        }
        return fresh==0 ? minutes : -1;
    }
}
```

Example / dry run: All initially rotten cells spread simultaneously. If one fresh orange is isolated by empty cells, return -1.

Important: Do not start a separate BFS for each rotten orange; multi-source BFS gives the correct simultaneous time.

3.4 Shortest path in binary matrix (8 directions)

What it solves: Find shortest path from top-left to bottom-right through open cells, with 8-direction movement.

Recognition rule / invariant: Use BFS because each move costs 1. Distance counts visited cells, so initialize start distance to 1.

Complexity: Time O(RC), space O(RC).

IntelliJ-ready Java implementation

```
import java.util.*;

public class ShortestPathBinaryMatrix {
    static int shortestPathBinaryMatrix(int[][] grid) {
        int n=grid.length;
        if(n==0 || grid[0][0]!=0 || grid[n-1][n-1]!=0) return -1;
        int[][] d={{1,0},{-1,0},{0,1},{0,-1},{1,1},{1,-1},{-1,1},{-1,-1}};
        int[][] dist=new int[n][n];
        for(int[] row:dist) Arrays.fill(row,-1);
        Queue<int[]> q=new ArrayDeque<>();
        q.offer(new int[]{0,0}); dist[0][0]=1;
```

```
        while(!q.isEmpty()){
            int[] x=q.poll();
            if(x[0]==n-1 && x[1]==n-1) return dist[x[0]][x[1]];
            for(int[] z:d){
                int nr=x[0]+z[0], nc=x[1]+z[1];
                if(nr>=0&&nr<n&&nc>=0&&nc<n&&grid[nr][nc]==0&&dist[nr][nc]==-1){
                    dist[nr][nc]=dist[x[0]][x[1]]+1;
                    q.offer(new int[]{nr,nc});
                }
            }
        }
        return -1;
    }
}
```

Example / dry run: Single open cell returns 1; blocked start or destination returns -1.

Important: Confirm 4-direction vs 8-direction movement and whether path length counts cells or edges.

4. Cycle detection and bipartite graph

4.1 Cycle in undirected graph (DFS)

What it solves: Detect whether an undirected graph contains a cycle.

Recognition rule / invariant: During DFS, a visited neighbor is a cycle only if it is not the current node's parent.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class UndirectedCycleDFS {
    static boolean dfs(int u, int parent, List<List<Integer>> g, boolean[] seen) {
        seen[u]=true;
        for(int v:g.get(u)){
            if(!seen[v]){
                if(dfs(v,u,g,seen)) return true;
            } else if(v!=parent) return true;
        }
        return false;
    }
    static boolean hasCycle(List<List<Integer>> g) {
        boolean[] seen=new boolean[g.size()];
        for(int i=0;i<g.size();i++)
            if(!seen[i] && dfs(i,-1,g,seen)) return true;
        return false;
    }
}
```

Example / dry run: Triangle 0—1—2—0 has a cycle. A simple chain does not.

Important: This parent method assumes a simple undirected graph. Parallel edges need careful handling (edge IDs), because a second edge to the parent can form a length-2 multigraph cycle.

4.2 Cycle in undirected graph (BFS)

What it solves: Detect an undirected cycle with queue traversal.

Recognition rule / invariant: Store parent of each discovered vertex; a visited neighbor other than parent indicates a cycle.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class UndirectedCycleBFS {
    static boolean hasCycle(List<List<Integer>> g) {
        int n=g.size();
        boolean[] seen=new boolean[n];
        int[] parent=new int[n];
        Arrays.fill(parent,-1);
        for(int s=0;s<n;s++) if(!seen[s]){
            Queue<Integer> q=new ArrayDeque<>();
            q.offer(s); seen[s]=true;
            while(!q.isEmpty()){
                int u=q.poll();
                for(int v:g.get(u)){
                    if(!seen[v]){
                        seen[v]=true; parent[v]=u; q.offer(v);
                    } else if(v!=parent[u]) return true;
                }
            }
        }
```

```
        return false;
    }
}
```

Example / dry run: Each BFS tree edge is ignored through parent; any additional visited connection signals a cycle.

Important: For multigraphs use edge IDs; parent vertex alone is not enough.

4.3 Cycle in directed graph (DFS colors)

What it solves: Detect a directed cycle.

Recognition rule / invariant: Use states 0=unvisited, 1=currently in recursion stack, 2=fully processed. An edge to state 1 is a back edge and proves a cycle.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class DirectedCycle {
    static boolean dfs(int u, List<List<Integer>> g, int[] state) {
        state[u]=1;
        for(int v:g.get(u)){
            if(state[v]==1) return true;
            if(state[v]==0 && dfs(v,g,state)) return true;
        }
        state[u]=2;
        return false;
    }
    static boolean hasCycle(List<List<Integer>> g) {
        int[] state=new int[g.size()];
        for(int i=0;i<g.size();i++)
            if(state[i]==0 && dfs(i,g,state)) return true;
        return false;
    }
}
```

Example / dry run: 0→1→2→0 reaches a node still in the recursion stack, so a cycle exists.

Important: Do not use the undirected parent-only rule for directed graphs. For very deep graphs, implement iterative color DFS or Kahn's algorithm.

4.4 Bipartite graph (BFS two-coloring)

What it solves: Determine whether vertices can be divided into two groups so every edge joins different groups.

Recognition rule / invariant: Color each component with 0/1; every edge must connect opposite colors.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class BipartiteGraph {
    static boolean isBipartite(List<List<Integer>> g) {
        int n=g.size();
        int[] color=new int[n];
        Arrays.fill(color,-1);
        for(int s=0;s<n;s++) if(color[s]==-1){
            Queue<Integer> q=new ArrayDeque<>();
            q.offer(s); color[s]=0;
            while(!q.isEmpty()){
                int u=q.poll();
                for(int v:g.get(u)){
                    if(color[v]==-1){ color[v]=1-color[u]; q.offer(v); }
                    else if(color[v]==color[u]) return false;
                }
            }
        }
        return true;
    }
}
```

Example / dry run: Any odd cycle (triangle, 5-cycle) makes a graph non-bipartite; every even cycle is bipartite.

Important: An isolated vertex is bipartite. Run from every component.

5. Topological sorting and dependency problems

5.1 Kahn's algorithm (BFS topological sort)

What it solves: Produce a linear ordering of a directed acyclic graph (DAG) so every u→v has u before v.

Recognition rule / invariant: Compute indegree; enqueue all zero-indegree vertices; remove them and decrement outgoing neighbors. If fewer than V vertices are output, a directed cycle exists.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class TopologicalSortKahn {
    static List<Integer> topo(List<List<Integer>> g) {
        int n=g.size(), count=0;
        int[] indeg=new int[n];
        for(int u=0;u<n;u++) for(int v:g.get(u)) indeg[v]++;
        Queue<Integer> q=new ArrayDeque<>();
        for(int i=0;i<n;i++) if(indeg[i]==0) q.offer(i);
        List<Integer> order=new ArrayList<>();
        while(!q.isEmpty()){
            int u=q.poll(); order.add(u); count++;
            for(int v:g.get(u)) if(--indeg[v]==0) q.offer(v);
        }
        return count==n ? order : Collections.emptyList();
    }
}
```

Example / dry run: If output has V vertices, a topological ordering exists. Multiple valid orders may exist.

Important: Only directed acyclic graphs have a topological ordering. Empty list here means cycle (for n>0).

5.2 Topological sort (DFS)

What it solves: Build topological order by finishing times.

Recognition rule / invariant: After visiting all outgoing neighbors, push the vertex onto a stack; reverse finishing order is topological order.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class TopologicalSortDFS {
    static void dfs(int u,List<List<Integer>> g,boolean[] seen,Deque<Integer> st){
        seen[u]=true;
        for(int v:g.get(u)) if(!seen[v]) dfs(v,g,seen,st);
        st.push(u);
    }
    static List<Integer> topo(List<List<Integer>> g){
        boolean[] seen=new boolean[g.size()];
        Deque<Integer> st=new ArrayDeque<>();
        for(int i=0;i<g.size();i++) if(!seen[i]) dfs(i,g,seen,st);
        return new ArrayList<>(st);
    }
}
```

Example / dry run: DFS finishing order gives a valid order only if the graph is acyclic.

Important: This compact version does not detect directed cycles. Use color states or Kahn's algorithm when cycle detection is required.

5.3 Course Schedule I

What it solves: Determine if all courses can be completed given prerequisite pairs [course, prerequisite].

Recognition rule / invariant: Create edge prerequisite→course; topologically process all courses. Completion count must equal numCourses.

Complexity: Time O(V+E), space O(V+E).

IntelliJ-ready Java implementation

```
import java.util.*;

public class CourseSchedule {
    static boolean canFinish(int n,int[][] prerequisites){
        List<List<Integer>> g=new ArrayList<>();
        for(int i=0;i<n;i++) g.add(new ArrayList<>());
        int[] indeg=new int[n];
        for(int[] p:prerequisites){
            int course=p[0], pre=p[1];
            g.get(pre).add(course);
            indeg[course]++;
        }
        Queue<Integer> q=new ArrayDeque<>();
```

```
        for(int i=0;i<n;i++) if(indeg[i]==0) q.offer(i);
        int done=0;
        while(!q.isEmpty()){
            int u=q.poll(); done++;
            for(int v:g.get(u)) if(--indeg[v]==0) q.offer(v);
        }
        return done==n;
    }
}
```

Example / dry run: Prerequisites 1←0 and 2←1 produce order 0,1,2. A cycle 0←1←0 makes completion impossible.

Important: Read pair direction carefully; the standard format [a,b] means b must be taken before a.

5.4 Course Schedule II

What it solves: Return one valid order to finish all courses, or empty array if impossible.

Recognition rule / invariant: Use Kahn's algorithm and return the order only if all vertices are processed.

Complexity: Time O(V+E), space O(V+E).

IntelliJ-ready Java implementation

```
import java.util.*;

public class CourseScheduleII {
    static int[] findOrder(int n,int[][] pre){
        List<List<Integer>> g=new ArrayList<>();
        for(int i=0;i<n;i++) g.add(new ArrayList<>());
        int[] in=new int[n];
        for(int[] p:pre){ g.get(p[1]).add(p[0]); in[p[0]]++; }
        Queue<Integer> q=new ArrayDeque<>();
        for(int i=0;i<n;i++) if(in[i]==0) q.offer(i);
        int[] ans=new int[n]; int k=0;
        while(!q.isEmpty()){
            int u=q.poll(); ans[k++]=u;
            for(int v:g.get(u)) if(--in[v]==0) q.offer(v);
        }
        return k==n ? ans : new int[0];
    }
}
```

Example / dry run: Independent courses can appear in any relative order; any valid topological ordering is accepted.

Important: Do not assume one unique answer.

6. Disjoint Set Union (DSU / Union-Find)

Purpose: Maintain groups under merge operations and answer whether two vertices are connected. It is central to Kruskal, redundant connection and accounts merge.

Formula / operations: find(x) returns representative; union(a,b) merges two sets. Path compression + union by size/rank gives amortized O(α(V)) per operation.

6.1 DSU implementation

What it solves: Efficiently merge components and test connectivity.

Recognition rule / invariant: Attach smaller tree under larger tree; compress paths during find.

Complexity: Amortized O(α(V)) per operation; space O(V).

IntelliJ-ready Java implementation

```
public class DSU {
    private final int[] parent, size;
    public DSU(int n) {
        parent=new int[n]; size=new int[n];
        for(int i=0;i<n;i++){ parent[i]=i; size[i]=1; }
    }
    public int find(int x) {
        if(parent[x]!=x) parent[x]=find(parent[x]);
        return parent[x];
    }
    public boolean union(int a,int b) {
        a=find(a); b=find(b);
        if(a==b) return false;
        if(size[a]<size[b]){ int t=a; a=b; b=t; }
        parent[b]=a; size[a]+=size[b];
        return true;
    }
    public boolean connected(int a,int b){ return find(a)==find(b); }
}
```

Example / dry run: Initially {0},{1},{2}. union(0,1) makes {0,1}; union(1,2) makes {0,1,2}; connected(0,2)=true.

Important: Do not compare parent[a] directly with parent[b]; call find on both.

6.2 Redundant Connection

What it solves: Find an edge that creates a cycle in a graph that started as a tree with one extra edge.

Recognition rule / invariant: Process edges in order; the first edge whose endpoints already share a DSU representative is redundant.

Complexity: Time O(E α(V)), space O(V).

IntelliJ-ready Java implementation

```
public class RedundantConnection {
    static class DSU {
        int[] p, sz;
        DSU(int n){p=new int[n+1];sz=new int[n+1];for(int i=0;i<=n;i++){p[i]=i;sz[i]=1;}}
        int find(int x){return p[x]==x?x:(p[x]=find(p[x]));}
        boolean union(int a,int b){
            a=find(a);b=find(b);if(a==b)return false;
            if(sz[a]<sz[b]){int t=a;a=b;b=t;}
            p[b]=a;sz[a]+=sz[b];return true;
        }
    }
    static int[] findRedundantConnection(int[][] edges){
        int n=edges.length; DSU d=new DSU(n);
        for(int[] e:edges) if(!d.union(e[0],e[1])) return e;
        return new int[0];
    }
}
```

Example / dry run: When processing [1,2], [1,3], [2,3], the third edge connects vertices already connected; return [2,3].

Important: This common version assumes labels 1..n and the problem's tree-plus-one-edge guarantee.

7. Minimum spanning tree (MST)

MST definition: A spanning tree of a connected, weighted, undirected graph with minimum possible sum of edge weights. Exactly V–1 edges.

Cut property: A minimum-weight edge crossing a cut is safe for some MST. This is the key correctness principle behind Kruskal and Prim.

7.1 Kruskal algorithm

What it solves: Build MST by accepting globally lightest edges that do not create a cycle.

Recognition rule / invariant: Sort edges by weight; DSU rejects edges whose endpoints are already connected.

Complexity: Time O(E log E); space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class KruskalMST {
    static class Edge {
        int u,v,w;
        Edge(int u,int v,int w){this.u=u;this.v=v;this.w=w;}
    }
    static class DSU {
        int[] p,s;
        DSU(int n){p=new int[n];s=new int[n];for(int i=0;i<n;i++){p[i]=i;s[i]=1;}}
        int find(int x){return p[x]==x?x:(p[x]=find(p[x]));}
        boolean union(int a,int b){
            a=find(a);b=find(b);if(a==b)return false;
            if(s[a]<s[b]){int t=a;a=b;b=t;}
            p[b]=a;s[a]+=s[b];return true;
        }
    }
    static long mstWeight(int n,List<Edge> edges){
        edges.sort(Comparator.comparingInt(e->e.w));
        DSU d=new DSU(n); long total=0; int used=0;
        for(Edge e:edges) if(d.union(e.u,e.v)){
            total+=e.w;
            if(++used==n-1) break;
        }
        return used==n-1?total:-1; // disconnected
    }
}
```

Example / dry run: Sort edges ascending; accept smallest edge that joins different components. Stop after V–1 accepted edges.

Important: Vertices are 0-indexed here. For disconnected graphs, returns -1 instead of an MST.

7.2 Prim algorithm (adjacency list + min-heap)

What it solves: Grow an MST from a starting vertex by adding the cheapest edge leaving the visited set.

Recognition rule / invariant: Min-heap stores candidate edges; ignore entries whose destination is already visited.

Complexity: Time O((V+E) log V) typical; space O(V+E).

IntelliJ-ready Java implementation

```
import java.util.*;

public class PrimMST {
    static class E { int to,w; E(int t,int w){to=t;this.w=w;} }
    static long mst(List<List<E>> g){
        int n=g.size(); if(n==0)return 0;
        boolean[] used=new boolean[n];
        PriorityQueue<int[]> pq=new PriorityQueue<>(Comparator.comparingInt(a->a[1]));
        pq.offer(new int[]{0,0});
        long sum=0; int count=0;
        while(!pq.isEmpty()){
            int[] cur=pq.poll(); int u=cur[0], w=cur[1];
            if(used[u])continue;
            used[u]=true; sum+=w; count++;
            for(E e:g.get(u)) if(!used[e.to]) pq.offer(new int[]{e.to,e.w});
        }
        return count==n?sum:-1;
    }
}
```

Example / dry run: At each step select the cheapest edge from visited to unvisited. The accepted edges never create a cycle.

Important: Graph must be undirected. For dense graphs, $O(V^2)$ matrix Prim can be preferable.

8. Shortest path algorithms: rules and formulas

Decision table:

Graph / task	Algorithm	Time
Unweighted / equal edge cost	BFS	O(V+E)
Weights only 0 or 1	0–1 BFS	O(V+E)
Non-negative weights	Dijkstra	O((V+E) log V)
Negative edges, no reachable negative cycle	Bellman–Ford	O(VE)
All pairs, moderate V	Floyd–Warshall	$O(V^3)$
DAG, weights may be negative	Topological relaxation	O(V+E)

Relaxation formula: dist[v] = min(dist[v], dist[u] + w(u,v)). Relaxation means improving a known distance using an edge.

8.1 Dijkstra (non-negative weights)

What it solves: Single-source shortest paths when every edge weight is >= 0.

Recognition rule / invariant: Extract the smallest tentative distance; relax neighbors. Skip stale priority-queue entries.

Complexity: O((V+E) log V) with adjacency list + binary heap; O(V+E) space.

IntelliJ-ready Java implementation

```
import java.util.*;

public class Dijkstra {
    static class Edge { int to,w; Edge(int t,int w){to=t;this.w=w;} }
    static long[] shortest(List<List<Edge>> g,int src){
        int n=g.size(); long INF=Long.MAX_VALUE/4;
        long[] dist=new long[n]; Arrays.fill(dist,INF); dist[src]=0;
        PriorityQueue<long[]> pq=new PriorityQueue<>(Comparator.comparingLong(a->a[1]));
        pq.offer(new long[]{src,0});
        while(!pq.isEmpty()){
            long[] cur=pq.poll(); int u=(int)cur[0]; long d=cur[1];
            if(d!=dist[u])continue;
            for(Edge e:g.get(u)){
                long nd=d+e.w;
                if(nd<dist[e.to]){
                    dist[e.to]=nd;
                    pq.offer(new long[]{e.to,nd});
                }
```

```
            }
        }
        return dist;
    }
}
```

Example / dry run: 0→1(4), 0→2(1), 2→1(2), 1→3(1): dist[1]=3 and dist[3]=4.

Important: Negative edges invalidate Dijkstra. Use long for distances and guard against overflow.

8.2 Bellman–Ford

What it solves: Single-source shortest paths with negative edge weights; detect reachable negative cycles.

Recognition rule / invariant: Relax every edge V−1 times. One more pass with any improvement means a reachable negative cycle.

Complexity: Time O(VE), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class BellmanFord {
    static class Edge { int u,v,w; Edge(int a,int b,int c){u=a;v=b;w=c;} }
    static long[] shortest(int n,List<Edge> edges,int src){
        long INF=Long.MAX_VALUE/4;
        long[] d=new long[n]; Arrays.fill(d,INF); d[src]=0;
        for(int i=1;i<n;i++){
            boolean changed=false;
            for(Edge e:edges) if(d[e.u]!=INF && d[e.u]+e.w<d[e.v]){
                d[e.v]=d[e.u]+e.w; changed=true;
            }
            if(!changed)break;
        }
        for(Edge e:edges)
            if(d[e.u]!=INF && d[e.u]+e.w<d[e.v])
                throw new IllegalStateException("Reachable negative cycle");
        return d;
    }
}
```

Example / dry run: Negative edge can improve a path. After at most V−1 edges, shortest simple paths are settled if no negative cycle is reachable.

Important: Negative cycle detection applies to cycles reachable from source. For detect-anywhere, initialize all distances to 0 or add a super-source.

8.3 Floyd–Warshall (all-pairs shortest paths)

What it solves: Compute shortest distance between every pair of vertices.

Recognition rule / invariant: Try each vertex k as an intermediate: dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]).

Complexity: Time $O(V^3)$, space $O(V^2)$.

IntelliJ-ready Java implementation

```
import java.util.*;

public class FloydWarshall {
    static final long INF=Long.MAX_VALUE/4;
    static long[][] allPairs(long[][] d){
        int n=d.length;
        for(int k=0;k<n;k++)
            for(int i=0;i<n;i++)
                for(int j=0;j<n;j++)
                    if(d[i][k]!=INF && d[k][j]!=INF)
                        d[i][j]=Math.min(d[i][j],d[i][k]+d[k][j]);
        return d;
    }
}
```

Example / dry run: Initialize d[i][i]=0, direct edge to its weight, and absent edges to INF. After all k, d[i][j] is shortest.

Important: If any d[i][i] < 0 after completion, a negative cycle exists. Do not add INF values without checking.

8.4 0–1 BFS

What it solves: Shortest path when each edge weight is either 0 or 1.

Recognition rule / invariant: Use a deque: push a 0-weight relaxation to front and a 1-weight relaxation to back.

Complexity: Time O(V+E), space O(V+E).

IntelliJ-ready Java implementation

```
import java.util.*;

public class ZeroOneBFS {
    static class E { int to,w; E(int t,int w){to=t;this.w=w;} }
    static int[] shortest(List<List<E>> g,int src){
        int n=g.size(), INF=1_000_000_000;
        int[] d=new int[n]; Arrays.fill(d,INF); d[src]=0;
        Deque<Integer> dq=new ArrayDeque<>(); dq.addFirst(src);
        while(!dq.isEmpty()){
            int u=dq.removeFirst();
            for(E e:g.get(u)){
                if(d[u]+e.w<d[e.to]){
                    d[e.to]=d[u]+e.w;
                    if(e.w==0)dq.addFirst(e.to); else dq.addLast(e.to);
                }
            }
        }
        return d;
    }
}
```

Example / dry run: Zero-cost edges are explored at the front; cost-1 edges at the back, preserving distance order.

Important: Every edge weight must be exactly 0 or 1. For arbitrary non-negative weights use Dijkstra.

8.5 Shortest path in a DAG

What it solves: Find shortest paths in a directed acyclic graph; negative edge weights are allowed.

Recognition rule / invariant: Topologically sort vertices, then relax outgoing edges once in topological order.

Complexity: Time O(V+E), space O(V+E).

IntelliJ-ready Java implementation

```
import java.util.*;

public class DAGShortestPath {
    static class E { int to,w; E(int t,int w){to=t;this.w=w;} }
    static long[] shortest(List<List<E>> g,int src){
        int n=g.size(); int[] in=new int[n];
        for(int u=0;u<n;u++)for(E e:g.get(u))in[e.to]++;
        Queue<Integer> q=new ArrayDeque<>();
        for(int i=0;i<n;i++)if(in[i]==0)q.offer(i);
        int[] order=new int[n];int k=0;
        while(!q.isEmpty()){
            int u=q.poll();order[k++]=u;
            for(E e:g.get(u))if(--in[e.to]==0)q.offer(e.to);
        }
        if(k<n)throw new IllegalArgumentException("Graph is not a DAG");
        long INF=Long.MAX_VALUE/4;
        long[] d=new long[n];Arrays.fill(d,INF);d[src]=0;
        for(int x:order)if(d[x]!=INF)
            for(E e:g.get(x))d[e.to]=Math.min(d[e.to],d[x]+e.w);
        return d;
    }
}
```

Example / dry run: Negative weights are safe because a DAG has no cycles; topological order ensures all incoming candidates are processed before a vertex is finalized.

Important: Do not apply DAG shortest path if a directed cycle exists.

9. High-frequency shortest-path implementation problems

9.1 Network Delay Time

What it solves: Time for a signal from source k to reach every node; return maximum shortest distance or -1.

Recognition rule / invariant: Build directed weighted graph, run Dijkstra, take maximum finite distance.

Complexity: O((V+E) log V), space O(V+E).

IntelliJ-ready Java implementation

```
import java.util.*;

public class NetworkDelayTime {
    static int networkDelayTime(int[][] times,int n,int k){
        List<List<int[]>> g=new ArrayList<>();
        for(int i=0;i<=n;i++)g.add(new ArrayList<>());
        for(int[] e:times)g.get(e[0]).add(new int[]{e[1],e[2]});
        long[] d=new long[n+1];Arrays.fill(d,Long.MAX_VALUE/4);d[k]=0;
        PriorityQueue<long[]> pq=new PriorityQueue<>(Comparator.comparingLong(a->a[1]));
        pq.offer(new long[]{k,0});
```

```
        while(!pq.isEmpty()){
            long[] x=pq.poll();int u=(int)x[0];long du=x[1];
            if(du!=d[u])continue;
            for(int[] e:g.get(u))if(du+e[1]<d[e[0]]){
                d[e[0]]=du+e[1];pq.offer(new long[]{e[0],d[e[0]]});
            }
        }
        long ans=0;
        for(int i=1;i<=n;i++){if(d[i]>=Long.MAX_VALUE/8)return -1;ans=Math.max(ans,d[i]);}
        return (int)ans;
    }
}
```

Example / dry run: Run Dijkstra from k. If any node is unreachable, answer -1; otherwise return the farthest shortest distance.

Important: Input labels are 1..n in this standard problem.

9.2 Cheapest Flights Within K Stops

What it solves: Find minimum flight cost from src to dst using at most k stops.

Recognition rule / invariant: Use Bellman–Ford-style bounded relaxations for at most k+1 edges; use a snapshot array per round so one round adds only one edge.

Complexity: Time O((K+1)E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class CheapestFlightsKStops {
    static int findCheapestPrice(int n,int[][] flights,int src,int dst,int k){
        long INF=Long.MAX_VALUE/4;
        long[] d=new long[n];Arrays.fill(d,INF);d[src]=0;
        for(int step=0;step<=k;step++){
            long[] next=d.clone();
            for(int[] f:flights){
                int u=f[0],v=f[1],w=f[2];
                if(d[u]!=INF)next[v]=Math.min(next[v],d[u]+w);
            }
            d=next;
        }
        return d[dst]==INF?-1:(int)d[dst];
    }
}
```

Example / dry run: k stops means at most k+1 flight edges. Snapshot prevents accidentally using multiple edges in one iteration.

Important: Off-by-one trap: run k+1 rounds, not k. Use long internally.

9.3 Path with Minimum Effort

What it solves: Minimize the maximum absolute height difference along a grid path.

Recognition rule / invariant: Use Dijkstra where path cost is the maximum edge difference encountered so far; relaxation is max(currentEffort, edgeDifference).

Complexity: O(RC log(RC)), space O(RC).

IntelliJ-ready Java implementation

```
import java.util.*;

public class MinimumEffortPath {
    static int minimumEffortPath(int[][] h){
        int r=h.length,c=h[0].length;
        int[][] d=new int[r][c];for(int[] row:d)Arrays.fill(row,Integer.MAX_VALUE);
        int[][] dirs={{1,0},{-1,0},{0,1},{0,-1}};
        PriorityQueue<int[]> pq=new PriorityQueue<>(Comparator.comparingInt(a->a[2]));
        d[0][0]=0;pq.offer(new int[]{0,0,0});
        while(!pq.isEmpty()){
            int[] x=pq.poll();int i=x[0],j=x[1],eff=x[2];
            if(eff!=d[i][j])continue;
            if(i==r-1&&j==c-1)return eff;
            for(int[] z:dirs){
                int ni=i+z[0],nj=j+z[1];
                if(ni>=0&&ni<r&&nj>=0&&nj<c){
                    int ne=Math.max(eff,Math.abs(h[i][j]-h[ni][nj]));
                    if(ne<d[ni][nj]){d[ni][nj]=ne;pq.offer(new int[]{ni,nj,ne});}
                }
            }
        }
        return -1;
    }
}
```

Example / dry run: Unlike normal shortest path, path score is the largest edge difference on that path, not the sum.

Important: Use Math.abs carefully if input values may be at int extremes; use long for differences in such cases.

10. Strongly connected components (SCC)

SCC: A maximal set of vertices in a directed graph where every vertex can reach every other vertex.

10.1 Kosaraju algorithm

What it solves: Find all SCCs in a directed graph using two DFS passes.

Recognition rule / invariant: First DFS on original graph to record finish order; reverse every edge; process vertices in decreasing finish order on the reversed graph.

Complexity: Time O(V+E), space O(V+E).

IntelliJ-ready Java implementation

```
import java.util.*;

public class KosarajuSCC {
    static void dfs1(int u,List<List<Integer>> g,boolean[] seen,Deque<Integer> st){
        seen[u]=true;
        for(int v:g.get(u))if(!seen[v])dfs1(v,g,seen,st);
        st.push(u);
    }
    static void dfs2(int u,List<List<Integer>> rg,boolean[] seen,List<Integer> comp){
        seen[u]=true;comp.add(u);
        for(int v:rg.get(u))if(!seen[v])dfs2(v,rg,seen,comp);
    }
    static List<List<Integer>> components(List<List<Integer>> g){
        int n=g.size();
        List<List<Integer>> rg=new ArrayList<>();
        for(int i=0;i<n;i++)rg.add(new ArrayList<>());
        for(int u=0;u<n;u++)for(int v:g.get(u))rg.get(v).add(u);
        boolean[] seen=new boolean[n];Deque<Integer> st=new ArrayDeque<>();
        for(int i=0;i<n;i++)if(!seen[i])dfs1(i,g,seen,st);
        Arrays.fill(seen,false);
        List<List<Integer>> ans=new ArrayList<>();
        while(!st.isEmpty()){
            int u=st.pop();
            if(!seen[u]){List<Integer> comp=new ArrayList<>();dfs2(u,rg,seen,comp);ans.add(comp);}
        }
        return ans;
    }
}
```

Example / dry run: Two vertices with paths both ways belong to one SCC. One-way reachability alone is not enough.

Important: Recursive DFS can overflow on deep graphs; iterative version may be needed for very large V.

10.2 Tarjan SCC: concept and implementation note

What it solves: Find SCCs in one DFS using discovery times and low-link values.

Recognition rule / invariant: Maintain discovery[u], low[u], an onStack flag, and a stack. When low[u]==disc[u], pop one complete SCC.

Complexity: O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class TarjanSCC {
    static int time=0;
    static int[] disc, low;
    static boolean[] on;
    static Deque<Integer> st;
    static List<List<Integer>> ans;

    static void dfs(int u,List<List<Integer>> g){
        disc[u]=low[u]=++time;
        st.push(u);on[u]=true;
        for(int v:g.get(u)){
            if(disc[v]==0){dfs(v,g);low[u]=Math.min(low[u],low[v]);}
            else if(on[v]) low[u]=Math.min(low[u],disc[v]);
        }
        if(low[u]==disc[u]){
            List<Integer> comp=new ArrayList<>();
            int v;
            do{v=st.pop();on[v]=false;comp.add(v);}while(v!=u);
            ans.add(comp);
        }
    }
    static List<List<Integer>> scc(List<List<Integer>> g){
```

```
        int n=g.size();time=0;
        disc=new int[n];low=new int[n];on=new boolean[n];
        st=new ArrayDeque<>();ans=new ArrayList<>();
        for(int i=0;i<n;i++)if(disc[i]==0)dfs(i,g);
        return ans;
    }
}
```

Example / dry run: An SCC root is found when low[u] equals disc[u]; pop until u.

Important: Static fields are reset on every call. For reusable production code, make them instance fields. The low-link update differs for DFS tree edges vs back edges: use low[child] and disc[on-stack neighbor].

11. Bridges and articulation points

Discovery time disc[u]: DFS entry timestamp. **Low[u]:** earliest discovery time reachable from u's DFS subtree using tree edges plus at most one back edge.

11.1 Bridges (critical connections)

What it solves: Find undirected edges whose removal increases the number of connected components.

Recognition rule / invariant: DFS tree edge u-v is a bridge if low[v] > disc[u].

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class Bridges {
    static int time;
    static int[] disc,low;
    static List<List<Integer>> ans;
    static void dfs(int u,int parent,List<List<Integer>> g){
        disc[u]=low[u]=++time;
        for(int v:g.get(u)){
            if(v==parent)continue;
            if(disc[v]==0){
                dfs(v,u,g);
                low[u]=Math.min(low[u],low[v]);
                if(low[v]>disc[u])ans.add(Arrays.asList(u,v));
            }else low[u]=Math.min(low[u],disc[v]);
        }
    }
    static List<List<Integer>> findBridges(List<List<Integer>> g){
        int n=g.size();time=0;disc=new int[n];low=new int[n];ans=new ArrayList<>();
        for(int i=0;i<n;i++)if(disc[i]==0)dfs(i,-1,g);
        return ans;
    }
}
```

Example / dry run: An edge in a cycle is not a bridge because an alternate route exists. A bridge has no alternate back path across its cut.

Important: For multigraphs, skip parent edge by edge ID, not parent vertex. Recursive depth can be O(V).

11.2 Articulation points

What it solves: Find vertices whose removal increases the number of connected components.

Recognition rule / invariant: Non-root u is articulation if some DFS child v has low[v] >= disc[u]. Root is articulation only if it has more than one DFS child.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class ArticulationPoints {
    static int time;
    static int[] disc,low;
    static boolean[] cut;
    static void dfs(int u,int parent,List<List<Integer>> g){
        disc[u]=low[u]=++time;int children=0;
        for(int v:g.get(u)){
            if(v==parent)continue;
            if(disc[v]==0){
                children++;dfs(v,u,g);
                low[u]=Math.min(low[u],low[v]);
                if(parent!=-1 && low[v]>=disc[u])cut[u]=true;
            }else low[u]=Math.min(low[u],disc[v]);
        }
```

```
        if(parent==-1 && children>1)cut[u]=true;
    }
    static List<Integer> find(List<List<Integer>> g){
        int n=g.size();time=0;disc=new int[n];low=new int[n];cut=new boolean[n];
        for(int i=0;i<n;i++)if(disc[i]==0)dfs(i,-1,g);
        List<Integer> out=new ArrayList<>();
        for(int i=0;i<n;i++)if(cut[i])out.add(i);
        return out;
    }
}
```

Example / dry run: Root special case is different: it must have at least two DFS tree children. Non-root uses low[child] >= disc[u].

Important: Undirected graph. Multigraph handling requires edge IDs.

12. More globally common graph interview implementations

12.1 Clone Graph

What it solves: Deep-copy a connected undirected graph with possible cycles.

Recognition rule / invariant: Use a map original node→clone. Create clone before traversing neighbors; map prevents infinite recursion and duplicate clones.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

class Node {
    public int val;
    public List<Node> neighbors;
    public Node() { val=0; neighbors=new ArrayList<>(); }
    public Node(int v) { val=v; neighbors=new ArrayList<>(); }
}
public class CloneGraph {
    private final Map<Node,Node> map=new HashMap<>();
    public Node cloneGraph(Node node){
        if(node==null)return null;
        if(map.containsKey(node))return map.get(node);
        Node copy=new Node(node.val);map.put(node,copy);
        for(Node nei:node.neighbors)copy.neighbors.add(cloneGraph(nei));
        return copy;
    }
}
```

Example / dry run: Cycles and self-loops are preserved because each original node is cloned only once.

Important: LeetCode-style Node class is included. If the platform provides Node, remove the duplicate class definition.

12.2 Word Ladder (shortest transformation length)

What it solves: Find minimum number of one-letter transformations from beginWord to endWord, using only dictionary words.

Recognition rule / invariant: Treat each word as a vertex; BFS explores transformations in increasing number of steps.

Complexity: Typical optimized wildcard-bucket approach $O(N \cdot L^2)$ preprocessing/search; space $O(N \cdot L)$.

IntelliJ-ready Java implementation

```
import java.util.*;

public class WordLadder {
    static int ladderLength(String begin,String end,List<String> words){
        Set<String> dict=new HashSet<>(words);
        if(!dict.contains(end))return 0;
        Queue<String> q=new ArrayDeque<>();q.offer(begin);
        Set<String> seen=new HashSet<>();seen.add(begin);
        int level=1;
        while(!q.isEmpty()){
            for(int sz=q.size();sz>0;sz--){
                String w=q.poll();
                if(w.equals(end))return level;
                char[] a=w.toCharArray();
                for(int i=0;i<a.length;i++){
                    char old=a[i];
                    for(char ch='a';ch<='z';ch++){
                        if(ch==old)continue;
                        a[i]=ch;String next=new String(a);
                        if(dict.contains(next)&&seen.add(next))q.offer(next);
                    }
                    a[i]=old;
                }
            }
            level++;
```

```
        }
        return 0;
    }
}
```

Example / dry run: Each BFS level represents one more word in the transformation sequence. Return 0 if no valid transformation exists.

Important: All words are assumed same length and lowercase English. The returned length counts words, not edges.

12.3 Accounts Merge (DSU)

What it solves: Merge accounts that share at least one email; names may be identical but do not merge on name alone.

Recognition rule / invariant: Assign an ID to each email; union emails belonging to the same account; group emails by DSU root.

Complexity: Near O(T α(T) + T log T), where T is total emails; space O(T).

IntelliJ-ready Java implementation

```
import java.util.*;

public class AccountsMerge {
    static class DSU {
        int[] p,s;
        DSU(int n){p=new int[n];s=new int[n];for(int i=0;i<n;i++){p[i]=i;s[i]=1;}}
        int find(int x){return p[x]==x?x:(p[x]=find(p[x]));}
        void union(int a,int b){a=find(a);b=find(b);if(a==b)return;
            if(s[a]<s[b]){int t=a;a=b;b=t;}p[b]=a;s[a]+=s[b];}
    }
    static List<List<String>> accountsMerge(List<List<String>> accounts){
        Map<String,Integer> id=new HashMap<>();
        Map<String,String> owner=new HashMap<>();
        int next=0;
        for(List<String> a:accounts)for(int i=1;i<a.size();i++){
            String e=a.get(i);
            if(!id.containsKey(e)){id.put(e,next++);owner.put(e,a.get(0));}
        }
        DSU d=new DSU(next);
        for(List<String> a:accounts)
            for(int i=2;i<a.size();i++)d.union(id.get(a.get(1)),id.get(a.get(i)));
        Map<Integer,List<String>> groups=new HashMap<>();
        for(String e:id.keySet())groups.computeIfAbsent(d.find(id.get(e)),x->new ArrayList<>()).add(e);
        List<List<String>> ans=new ArrayList<>();
        for(List<String> es:groups.values()){
            Collections.sort(es);List<String> row=new ArrayList<>();
            row.add(owner.get(es.get(0)));row.addAll(es);ans.add(row);
        }
        return ans;
    }
}
```

Example / dry run: Shared email links two accounts; DSU merges their email components. Sort emails inside each final group.

Important: Assumes each account has at least one email. Name is taken from the owner map; problem guarantees merged accounts share the same person's name.

12.4 Is Graph Bipartite? (DFS alternative)

What it solves: Two-color graph using DFS instead of BFS.

Recognition rule / invariant: Assign opposite color to each neighbor; conflict means odd cycle.

Complexity: Time O(V+E), space O(V).

IntelliJ-ready Java implementation

```
import java.util.*;

public class BipartiteDFS {
    static boolean dfs(int u,List<List<Integer>> g,int[] color){
        for(int v:g.get(u)){
            if(color[v]==-1){color[v]=1-color[u];if(!dfs(v,g,color))return false;}
            else if(color[v]==color[u])return false;
        }
        return true;
    }
    static boolean check(List<List<Integer>> g){
        int[] color=new int[g.size()];Arrays.fill(color,-1);
        for(int i=0;i<g.size();i++)if(color[i]==-1){
            color[i]=0;if(!dfs(i,g,color))return false;
        }
        return true;
    }
}
```

Example / dry run: Use either BFS or DFS; both enforce the same two-coloring constraint.

Important: Recursive DFS may overflow for huge graphs.

13. Eulerian path and circuit rules

Eulerian path: traverses every edge exactly once. **Eulerian circuit:** traverses every edge exactly once and returns to start. This is different from Hamiltonian path, which visits every vertex once.

Undirected connected graph (ignoring isolated vertices)

Euler circuit exists iff every vertex has even degree. Euler trail (not necessarily closed) exists iff exactly 0 or 2 vertices have odd degree.

Directed graph

For an Euler circuit: every vertex has indegree=outdegree and all nonzero-degree vertices belong to one connected component in the appropriate directed sense. For an Euler trail: one vertex has outdegree=indegree+1, one has indegree=outdegree+1, all others balanced, plus connectivity conditions.

13.1 Hierholzer algorithm (undirected Euler circuit)

What it solves: Construct an Euler circuit when all nonzero-degree vertices are connected and all degrees are even.

Recognition rule / invariant: Walk unused edges with a stack; when a vertex has no unused edge, add it to the circuit while backtracking.

Complexity: Time O(V+E), space O(V+E).

IntelliJ-ready Java implementation

```
import java.util.*;

public class EulerCircuit {
    static List<Integer> circuit(List<List<Integer>> g,int start){
        int n=g.size();
        List<Deque<Integer>> adj=new ArrayList<>();
        for(int i=0;i<n;i++)adj.add(new ArrayDeque<>(g.get(i)));
        Deque<Integer> st=new ArrayDeque<>();List<Integer> path=new ArrayList<>();
        st.push(start);
        while(!st.isEmpty()){
            int u=st.peek();
            if(!adj.get(u).isEmpty()){
                int v=adj.get(u).removeLast();
                adj.get(v).removeFirstOccurrence(u);
                st.push(v);
            }else path.add(st.pop());
        }
        Collections.reverse(path);
        return path;
    }
}
```

Example / dry run: Every edge is used once; circuit contains E+1 vertices in its output list.

Important: This simple adjacency-removal template is for undirected graphs without parallel-edge ambiguity. For multigraphs, use edge IDs and a used[] array. Validate degree/connectivity before calling.

14. Formula sheet and coding rules

Topic	Rule / formula
BFS shortest distance	dist[v] = dist[u] + 1
Weighted relaxation	dist[v] = min(dist[v], dist[u] + w(u,v))
Dijkstra	Only non-negative edge weights
Bellman–Ford	V−1 relaxation rounds; one extra detects reachable negative cycle
Floyd–Warshall	d[i][j] = min(d[i][j], d[i][k] + d[k][j])
DAG shortest path	Topological order + one relaxation pass
MST edge count	A spanning tree has exactly V−1 edges
Kruskal	Sort edges by weight + DSU

Topic	Rule / formula
Prim	Cheapest edge from current tree to outside
Bridge	low[child] > disc[parent]
Articulation non-root	low[child] >= disc[u]
Articulation root	More than one DFS tree child
Bipartite	No odd cycle; 2-coloring possible
Topological order	Every directed edge u→v has pos[u] < pos[v]
Euler circuit, undirected	All nonzero-degree vertices connected; every degree even
Euler trail, undirected	Connected nonzero-degree part; exactly 0 or 2 odd degrees

Java implementation rules for interviews

• Use List> for adjacency lists; add reverse edge only for undirected graphs.

• Use ArrayDeque for Queue/Deque/Stack behavior; avoid legacy Stack class.

• PriorityQueue is min-heap by default; use Comparator.comparingLong or Integer.compare safely.

• Use long for distances, sums and weights when constraints can exceed int.

• Initialize unreachable distance with INF (e.g. Long.MAX_VALUE/4), not Long.MAX_VALUE if adding values.

• Mark visited at enqueue time in BFS.

• For disconnected graphs, loop over all vertices to start traversal in every component.

• Check indexing: 0-based vs 1-based.

• State graph direction and edge weight constraints before selecting algorithm.

• Use iterative DFS if recursion depth may reach 100,000+.

• Do not use Dijkstra for negative edges; do not use MST for directed shortest paths.

• For multigraphs, parent vertex is insufficient in bridge/cycle algorithms; track edge IDs.

15. Global interview problem bank

Traversal / components / matrix

■ Binary Tree? (tree traversal is separate; graph traversal fundamentals)

■ Number of Provinces

■ Number of Connected Components in an Undirected Graph

■ Flood Fill

■ Number of Islands

■ Max Area of Island

■ Surrounded Regions

■ Pacific Atlantic Water Flow

■ Rotting Oranges

■ 01 Matrix

■ Shortest Path in Binary Matrix

■ Nearest Exit from Entrance in Maze

■ Keys and Rooms

■ Reorder Routes to Make All Paths Lead to the City Zero

Cycle / coloring / topology

- Detect Cycle in an Undirected Graph
- Detect Cycle in a Directed Graph
- Is Graph Bipartite?
- Possible Bipartition
- Course Schedule
- Course Schedule II
- Find Eventual Safe States
- Alien Dictionary
- Minimum Height Trees
- Sequence Reconstruction

DSU / MST / connectivity

- Redundant Connection
- Redundant Connection II (directed; specialized logic)
- Accounts Merge
- Number of Operations to Make Network Connected
- Most Stones Removed with Same Row or Column
- Number of Islands II (dynamic connectivity)
- Min Cost to Connect All Points
- Connecting Cities With Minimum Cost
- Optimize Water Distribution in a Village
- Critical Connections in a Network

Shortest paths

- Network Delay Time
- Cheapest Flights Within K Stops
- Path With Minimum Effort
- Path with Maximum Probability
- Find the City With the Smallest Number of Neighbors at a Threshold Distance
- Shortest Path in DAG
- Shortest Path in Weighted Undirected Graph
- Word Ladder
- Word Ladder II (all shortest paths; BFS + backtracking)
- Evaluate Division (weighted graph traversal)
- Snakes and Ladders
- Open the Lock

Advanced graph structures

- Strongly Connected Components (Kosaraju)
- Strongly Connected Components (Tarjan)
- Bridges in a Graph

■ Articulation Points

■ Eulerian Path / Itinerary Reconstruction

■ Reconstruct Itinerary (Hierholzer + lexical order)

■ Clone Graph

■ All Paths From Source to Target

■ Critical and Pseudo-Critical Edges in MST

■ Swim in Rising Water

Further advanced topics to study after this guide: maximum flow (Edmonds–Karp/Dinic), bipartite matching (Kuhn/Hopcroft–Karp), minimum-cost flow, centroid decomposition, heavy-light decomposition, LCA/binary lifting, tree DP, rerooting DP, A* search, Johnson's algorithm, and planar graph algorithms.

16. IntelliJ IDEA project setup and submission checklist

• Open IntelliJ IDEA → New Project → Java → select JDK 17 or later → Create.

• Create one Java class per code block: right-click src → New → Java Class → use the exact class name shown.

• Paste imports at the top. Keep only one public top-level class per .java file.

• If testing locally, add a public static void main(String[] args) method and construct a small graph.

• Run using the green triangle beside main(). For online judges, remove main if the platform supplies its own driver and match the required class/method signature.

• Check whether platform expects int, long, int[], List, or a custom Pair/Node class.

• Before submission test: empty graph, one vertex, disconnected graph, self-loop, duplicate edges, cycle, negative edge (if allowed), and maximum constraints.

Minimal IntelliJ smoke test

Save as Main.java; the BFS class must also be present in the project under BFS.java.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int n = 4;
        List<List<Integer>> g = new ArrayList<>();
        for (int i = 0; i < n; i++) g.add(new ArrayList<>());

        // Undirected edges: 0-1, 1-2, 0-3
        g.get(0).add(1); g.get(1).add(0);
        g.get(1).add(2); g.get(2).add(1);
        g.get(0).add(3); g.get(3).add(0);

        System.out.println(BFS.bfs(g, 0));
    }
}
```

17. How to master graph interviews

• First classify: directed/undirected, weighted/unweighted, connected/disconnected, negative weights or not.

• Draw a 5–6 node example and manually simulate the chosen algorithm.

• Write the invariant: what does visited, dist, indegree, parent, low-link, or DSU representative mean?

• Implement from memory in IntelliJ, then test edge cases.

• Explain why the algorithm is correct, not only how the code runs.

• State time and space complexity in terms of V and E.

• Practice problem variations: return boolean, count, order, path itself, or minimum cost.

Final note: This is a broad core interview guide, not a guarantee that every company question is covered. For top-tier interviews, pair these templates with problem solving, proof explanations, and timed implementation practice.